Practical 12

Implement a simple AI learning model such as k-NN, decision tree, Naive Bayes, or perceptron and compare results on a toy dataset.

Objective

To implement and compare three simple supervised learning models — k-Nearest Neighbours, Decision Tree, and Gaussian Naive Bayes — on the classic Iris toy dataset, and to evaluate and compare their classification accuracy and confusion matrices.

Procedure

1. Load the Iris dataset (150 samples, 4 numeric features, 3 balanced classes: setosa, versicolor, virginica) from scikit-learn.
2. Split the data into training (70%) and testing (30%) sets using a stratified split so all three classes are proportionally represented.
3. Standardize the features (zero mean, unit variance) for the distance-based k-NN model using StandardScaler.
4. Train three models on the training set: k-NN (k=5) on the scaled features, a depth-limited Decision Tree (max_depth=3), and Gaussian Naive Bayes, both on the raw features.
5. Predict on the held-out test set for each model and compute accuracy and a full confusion matrix.
6. Compare the three models' test accuracy side by side and identify the best-performing model on this split.

Program / Code

```
# ================================================================
# PRACTICAL 12
# Simple AI Learning Models - k-NN, Decision Tree, Naive Bayes
# Compared on the Iris toy dataset
# ================================================================

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target
target_names = [str(t) for t in iris.target_names]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "k-NN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "Naive Bayes": GaussianNB(),
}

print("=" * 65)
print("   SIMPLE AI LEARNING MODELS ON THE IRIS TOY DATASET")
print("=" * 65)
print(f"\nDataset       : {X.shape[0]} samples, {X.shape[1]} features, {len(target_names)} classes")
print(f"Classes       : {list(target_names)}")
print(f"Train / Test  : {len(X_train)} / {len(X_test)} (stratified 70/30 split)")

results = {}

for name, model in models.items():
    if name == "k-NN (k=5)":
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    results[name] = (acc, cm)

    print("\n" + "-" * 65)
    print(f"Model: {name}")
    print(f"  Accuracy : {acc:.4f}")
    print("  Confusion Matrix (rows=true, cols=predicted):")
    print("           " + "  ".join(f"{t[:4]:>6}" for t in target_names))
    for i, row in enumerate(cm):
        print(f"  {target_names[i][:4]:>6} " + "  ".join(f"{v:>6}" for v in row))

print("\n" + "=" * 65)
print("SUMMARY - Accuracy Comparison")
print("-" * 65)
for name, (acc, _) in results.items():
    bar = "#" * int(acc * 40)
    print(f"  {name:<16}: {acc:.4f}  {bar}")

best_model = max(results, key=lambda k: results[k][0])
print(f"\nBest performing model on this split: {best_model} ({results[best_model][0]:.4f})")
print("=" * 65)
```

Output

```
=================================================================
   SIMPLE AI LEARNING MODELS ON THE IRIS TOY DATASET
=================================================================
```

```
Dataset       : 150 samples, 4 features, 3 classes
Classes       : ['setosa', 'versicolor', 'virginica']
Train / Test  : 105 / 45 (stratified 70/30 split)

------------------------------------------------------------------
Model: k-NN (k=5)
  Accuracy : 0.9111
  Confusion Matrix (rows=true, cols=predicted):
             seto    vers    virg
    seto     15       0       0
    vers      0      15       0
    virg      0       4      11

------------------------------------------------------------------
Model: Decision Tree
  Accuracy : 0.9778
  Confusion Matrix (rows=true, cols=predicted):
             seto    vers    virg
    seto     15       0       0
    vers      0      14       1
    virg      0       0      15

------------------------------------------------------------------
Model: Naive Bayes
  Accuracy : 0.9111
  Confusion Matrix (rows=true, cols=predicted):
             seto    vers    virg
    seto     15       0       0
    vers      0      14       1
    virg      0       3      12

==================================================================
SUMMARY - Accuracy Comparison
------------------------------------------------------------------
  k-NN (k=5)      : 0.9111  ###################################
  Decision Tree   : 0.9778  #####################################
  Naive Bayes     : 0.9111  ###################################

Best performing model on this split: Decision Tree (0.9778)
==================================================================
```

Conclusion

All three models performed strongly on the Iris test set, reflecting how linearly-separable Iris's classes are: the Decision Tree (max_depth=3) achieved the highest accuracy at 97.8%, with only a single versicolor sample confused for virginica, while k-NN and Gaussian Naive Bayes tied at 91.1% accuracy, both struggling with the same handful of versicolor/virginica boundary cases (these two species overlap slightly in petal measurements, unlike setosa which every model separated perfectly). This experiment illustrates a general lesson in applied machine learning: even very simple, easily-explainable models can achieve high accuracy on well-behaved toy datasets, and comparing multiple model families on the same

split is a quick, cheap way to sanity-check which inductive bias (distance-based, rule/split-based, or probabilistic) best matches a given dataset before scaling up to more complex, real-world data.